

DSD 2025 - Final Project

Report

강의 교수자: 차세대반도체학과 송인철 교수

5팀

전자정보공학부 IT융합전공 20211513 김동준

전자정보공학부 전자공학전공 20210316 최 영

전자정보공학부 IT융합전공 20212996 허준범

목차

목차.....	1
1. 팀 구성원 역할 분담.....	3
2. 초기 구현 계획.....	4
3. 최종 구현 시스템 개요.....	7
4. 구현 설명.....	8
5. 시스템 흐름.....	17
6. 결과 및 검증.....	18
7. 구조 개선안.....	24
8. 개인 project 소감.....	25
9. 별첨.....	27

1. 팀 구성원 역할 분담

1.1. module별 역할 분담표

module 설계	Module	TB
FirTop	공동	
FIR_FSM_Controller	최영	김동준
SRAM_Controller_4Parallel	허준범	최영
SeqMacController	김동준	최영
SeqMac	김동준	허준범
AdderWithSat	최영	허준범
saturation_fixer	김동준	생략 (과제1 재사용)

1.2. 기타 역할 분담

- Team leader: 김동준
- 서류 작업: 공동

2. 초기 구현 계획

2.1. concept

가. Controller Unit

- **Multiple FSM**
4 state FSM + 10 cycle counter at MAC
- **Mem access controller**
upper address mux -> SPRAM # -> write enable 신호 생성
- **MAC controller**
iNumOfCoeff[5:0]를 기준으로 판단 -> MAC enable 신호 생성
- **bus fabric**
memory write 관련 data 라인을 bus 형태로 모든 SPRAM이 공유

나. Mac unit 사용

- MAC 구조를 활용
- 여러 clock cycle을 이용해 비용이 비싼 multiplier 개수 절약

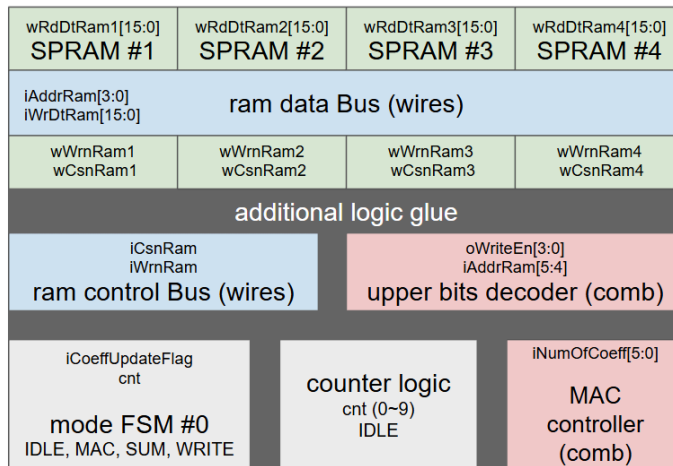
다. saturation handling adder 사용

- 발생하는 overflow/underflow를 관리하여 안정적인 값 생성.

2.2. 모듈 초안

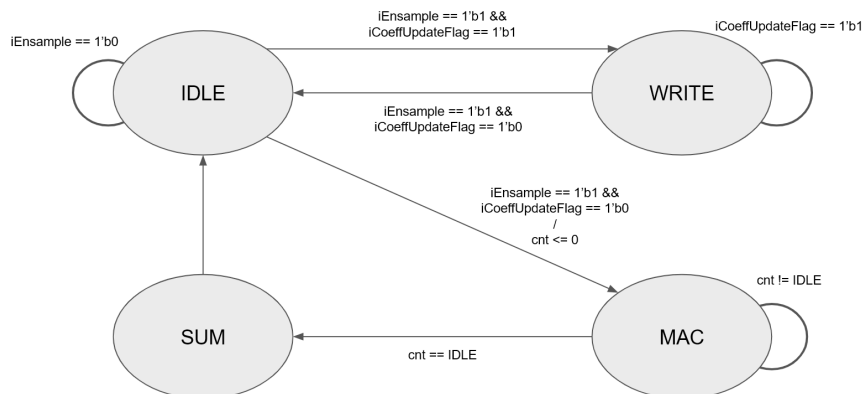
가. 컨트롤 유닛

Controller concept



- write data 신호들은 bus로 공유
- controller가 write할 SPSRAM을 선택
- 각 logic을 연결하기 위한 logic glue

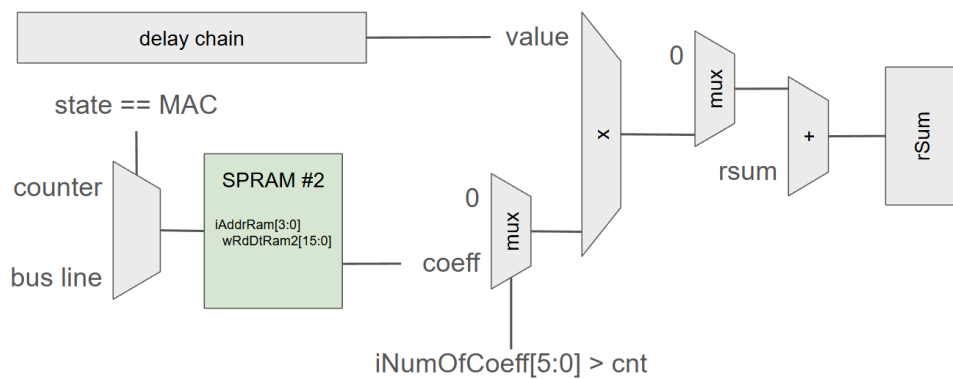
Main FSM



- Main FSM은 큰 전체 mode를 관장
- Main FSM은 iCcoeffUpdateFlag와 counter 값으로 state transition.

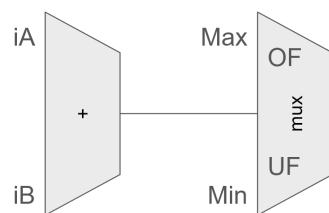
나. 실행 유닛

MAC unit



- counter logic은 sequential mac의 동작을 관장, ram access address 생성
- state가 MAC이 되면 counter가 동작, SPRAM은 read 상태
- count가 끝나면 IDLE로 가며 main FSM의 mode를 전환
- iNumOfCoeff[5:0]에 따라서 필터 길이 밖이라고 판단, 무시

Saturation handling summation unit



- summation 과정에서 입력의 부호를 기준으로, 출력 부호가 변하는지 판단
- (+) , (+) => (-) : saturation(overflow) => max
- (-) , (-) => (+) : saturation(underflow) => min

3. 최종 구현 시스템 개요

본 시스템은 4-Parallel SRAM 구조를 가진 재구성 가능한 Reconfigurable FIR 필터이다.

3.1. FIR 필터 기본 사양

- Clock speed: 12MHz
- sampling Frequency: 600KHz
- input: signed 3bit
- output: signed 16bit
- 가변 길이 대응 (최대 63 ~ 최저 1, 홀수)
- 32개의 SPSRAM, 32개의 coefficient 저장 및 수정 가능
(Folded Structure, k개 저장, 2k+1 길이 사용.)
- Group delay : 31
- Extra delay : 2 (by delayed input)

3.2. 주요 특징

- Reconfigurable Coefficients SRAM: 외부 인터페이스를 통해 필터 계수 실시간 업데이트 가능. a_i 계수를 변경 가능.
- 필터 업데이트 / 연산 모드 간의 전환 가능
- 병렬 MAC: 4개의 SRAM 과 MAC 유닛을 조합, 병렬 구동.
- Saturation Arithmetic: 오버/언더 플로우 방지를 위한 Saturation 로직 적용.
- Folded Structure: FIR의 대칭성을 활용하여 중복 저장 및 연산을 제거.

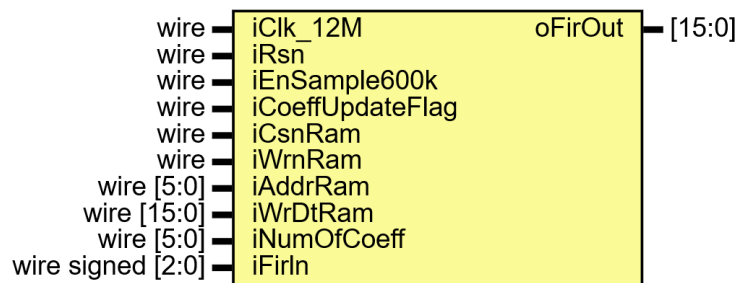
4. 구현 설명

4.1. 최상위 모듈

가. FirTop

전체 필터 시스템의 최상위 모듈로, 제어부(Control Plane)와 데이터 처리부(Data Plane)를 통합한다. 모든 하위 모듈을 불러와 통합한다.

Diagram



- 기능: 외부 제어 신호 수신, 하위 모듈 간 데이터 라우팅, 최종 필터 출력 생성.
- 구조: 63-tap Delay Line (Shift Register) 내장.

포트 정의

포트 이름	방향	타입	설명
System			
iClk_12M	Input	wire	시스템 클럭 (12MHz)
iRsn	Input	wire	리셋 신호 (Active Low)
Control			
iEnSample600k	Input	wire	샘플링 트리거 (600kHz)
iCoeffUpdateFlag	Input	wire	계수 업데이트 모드 플래그
iNumOfCoeff	Input	wire [5:0]	유효 필터 계수 개수
Memory Interface			
iCsnRam	Input	wire	전체 SRAM 제어신호
iWrnRam	Input	wire	전체 SRAM 제어신호
iAddrRam	Input	wire [5:0]	쓰기용 SRAM 주소
iWrDtRam	Input	wire [15:0]	쓰기용 Data
Data Path			
iFirIn	Input	wire signed [2:0]	입력 데이터 샘플
oFirOut	Output	wire signed [15:0]	필터 연산 최종 출력

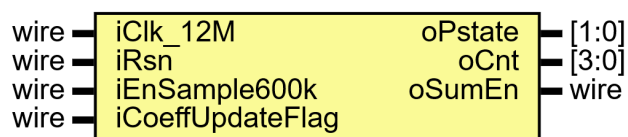
4.2. 컨트롤 유닛

시스템의 상태 흐름과 연산 타이밍을 제어하는 모듈 그룹. 상태를 관장하는 FSM과 외부의 복잡한 제어신호를 생성하는 combinational logic들로 이루어져있음.

가. FIR_FSM_Controller

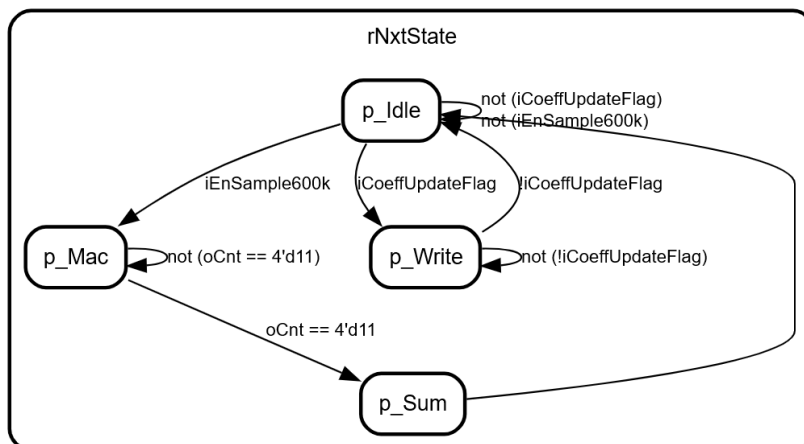
시스템의 전체 역할을 정하는 메인 FSM.

Diagram



- 상태 천이 (State Transition):
 - **IDLE**: 대기 상태. 샘플링 신호(iEnSample600k)나 업데이트 플래그 대기.
 - **WRITE**: 계수 업데이트 모드. 외부 데이터가 SRAM에 기록됨.
 - **MAC**: 연산 수행. 4개의 MAC 유닛이 병렬로 곱셈-누적 수행.
 - **SUM**: 최종 합산. 각 MAC 유닛의 결과를 더하여 출력 생성.
- 주요 출력: 현재 상태(oPstate), 연산 카운터(oCnt), 최종 합산 활성화(oSumEn).

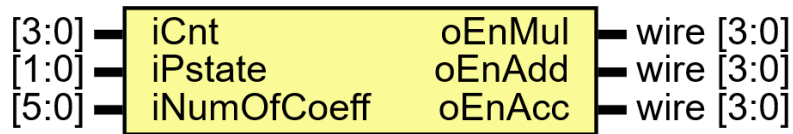
State machines



나. SeqMacController

FSM의 상태에 따라 4개의 병렬 MAC 유닛을 제어.

Diagram

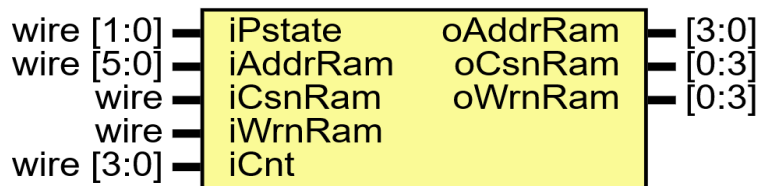


- 기능
 - 필터 계수를 받아, 대칭성을 고려하여 각 MAC 유닛의 Enable 신호(Mul, Add, Acc)를 동적으로 생성.
 - mac 연산 과정(p_Mac)에서 iCnt와 iNumOfCoeff를 비교하여 현재 시점에 동작해야 할 MAC 유닛만 선별적으로 활성화하여 SRAM에서 나오는 쓰레기 값을 계산하지 않게 값 보호.

다. SRAM_Controller_4Parallel

FSM의 상태와 외부 제어신호, 주소에 따라 4개의 SRAM 뱅크에 대한 접근 권한을 중재(Arbitration).

Diagram



- 내부 구조
 - Address mux : write/read에 따라서 sram address를 icnt와 write address간 결정
 - Arbitrator : write mode에서는 address 상위 2bit를 해석하여 적절한 SRAM에 write 신호 생성

- 기능

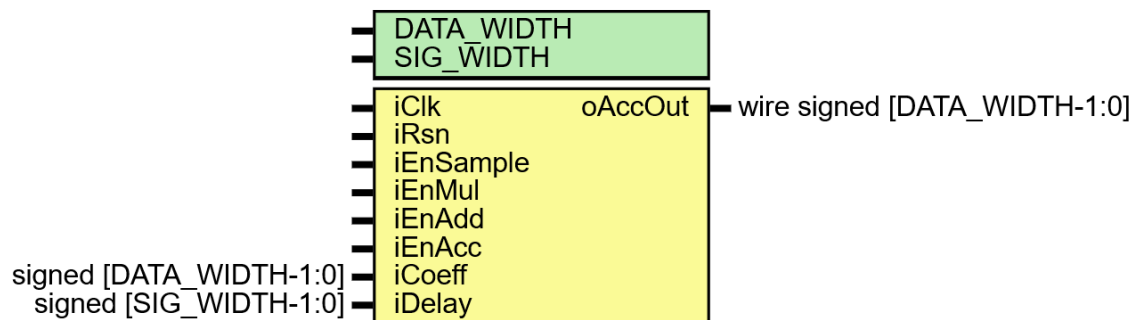
- 메인 FSM의 state에 따라 기본적인 모드를 결정
- 외부의 SRAM의 신호를 준수하여 SRAM 제어 신호를 생성
- **MAC Mode:** 내부 카운터(iCnt)를 주소로 사용하여 4개 बैं크에서 동시에 계수를 읽음 (p_Mac).
- **Write Mode:** 외부 주소(iAddrRam)를 입력받아 특정 बैं크, 특정 주소에 계수를 update함 (p_Write).

4.3. 실행 유닛

실제 데이터의 저장, 연산, 보정을 담당하는 모듈 그룹.

가. SeqMac

Diagram

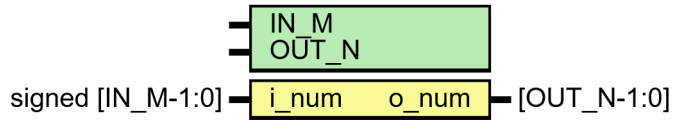


핵심 연산 유닛으로, 제어 신호와 clock에 따라 입력 샘플과 계수의 곱셈 및 누적을 수행.

- 기능
 - **Mul:** 입력 데이터(Delay)와 계수(Coeff) 곱셈 시행
 - **Add:** 현재 값과 곱셈 값 합 시행
 - **Acc:** 결과 저장
 - **Reset:** iEnSample이 1일때 ACC 초기화 진행.
- 내부구조
 - 비트 폭이 확장된 ACC를 사용 (계산 과정에서 값이 잠시 비트폭을 넘어서는 경우를 대응하여 정확도 향상)
 - 출력단에서 saturation_fixer를 통해 비트 폭을 맞춤.(saturation 출력 보장)

나. saturation_fixer

Diagram

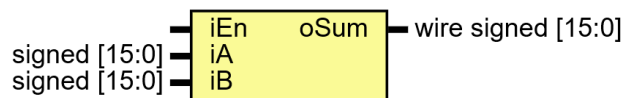


비트 폭 변환 시 데이터 무결성을 보장하는 모듈.

- **기능:** 입력값이 출력 비트 폭의 표현 범위를 벗어나면 자동으로 크기의 최대/최소값으로 클리핑(Clipping)하여 오버플로우 왜곡 방지.

다. AdderWithSat (Adder with Saturation)

Diagram

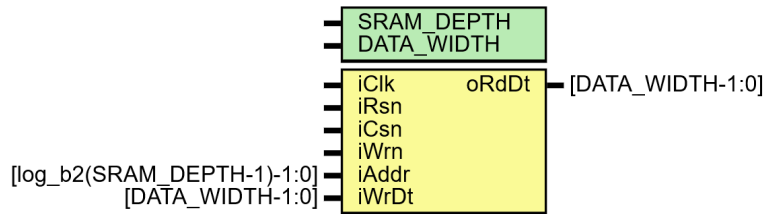


MAC 유닛들의 결과를 합산하는 가산기.

- **기능:** 두 개의 16비트 입력을 더함.
 - **Saturation:** 덧셈 결과가 16비트 범위(-32768 ~ 32767)를 초과할 경우, 자동으로 최대/최소값으로 클리핑(Clipping)하여 오버플로우 왜곡 방지.

라. SpSram (Single-Port SRAM)

Diagram



필터 계수를 저장하는 메모리 뱅크.

- 구성: 16 Depth 32 Bit (파라미터 설정 가능)
- 동작: 동기식 Read/Write 동작. 초기화 시 모든 값 0으로 리셋.

4.4. [붙임.] 상위 5개 SRAM 과 Delay chain간 wiring

MAC Unit	address	wTap Index	Delay Chain Input A	Delay Chain Input B	Description
MAC0	00_0000	wTap[0]	rDelayCain [31]	N/A	Center Tap (Middle)
MAC0	00_0001	wTap[1]	rDelayCain [32]	rDelayCain [30]	Symmetric Pair 1
MAC0	00_0010	wTap[2]	rDelayCain [33]	rDelayCain [29]	Symmetric Pair 2
MAC0	00_0011	wTap[3]	rDelayCain [34]	rDelayCain [28]	Symmetric Pair 3
MAC0	00_0100	wTap[4]	rDelayCain [35]	rDelayCain [27]	Symmetric Pair 4

(전체는 별도 첨부)

5. 시스템 흐름

5.1. 모드별 시스템 동작 순서

가. 필터 연산

1. 입력: iFirIn 데이터가 FirTop 내부의 Shift Register에 저장됨.
2. 제어: FIR_FSM_Controller가 MAC 연산(p_Mac)을 시작.
3. 인출: SRAM_Controller가 4개의 SpSram에서 계수(wCoeff)를 병렬적으로 인출.
4. 연산: SeqMacController의 En 신호에 따라 4개의 SeqMac이 연산 수행.
5. 합산: AdderWithSat 트리(Add01, Add23 -> AddTotal)를 통해 최종 결과 도출.
6. 출력: oFirOut으로 최종 필터링 데이터 출력.

나. 필터 업데이트

1. 입력: setup Flag 설정
2. 제어: FIR_FSM_Controller가 저장(p_Write)을 시작.
3. 저장: SRAM_Controller가 주소에 맞게 4개의 SpSram을 arbitration
4. 복귀: setup Flag가 돌아오면 FIR_FSM_Controller가 IDLE로 돌아가 연산 대기

6. 결과 및 검증

6.1. 검증 계획

ReConf_FirFilter_tb 파일은 FirTop 모듈을 최상위 DUT로 설정하여 재구성 가능 FIR 필터가 의도한 기능을 수행하는지 검증하기 위한 시뮬레이션 환경을 구성함. 실제 하드웨어 동작 흐름을 모방하여 입력 신호를 인가하고 필터의 반응을 관측함.

가. 테스트벤치의 구조

- DUT 인스턴스
 - FirTop 모듈을 uut라는 이름으로 인스턴스화하여 테스트 대상과 연결함.
- 클럭 및 리셋 생성
 - 12MHz 클럭(iClk_12M)과 시스템 초기화를 위한 리셋(iRsn)을 생성하여 DUT에 공급.
- 테스트 시나리오 호출

나. 테스트벤치 시나리오

- 시나리오 기반 검증
 - initial 블록 내에서 여러 task를 순차적으로 호출하며 다음과 같은 절차를 수행함.
 - 신호 초기화(initialize_signals): 모든 입력 신호를 안정적인 초기값으로 설정함.
 - 리셋 인가(apply_reset): DUT를 예측 가능한 상태로 초기화하도록 리셋을 인가함.
 - 계수 로딩(load_coefficients): 계수 업데이트 모드를 활성화해 SRAM에 필터 계수를 기록함.
 - 필터 동작 테스트(run_fir_test): 필터 실행 모드에서 입력을 인가하고 출력을 관측함.
 - 필터 sat 동작 테스트(fir_saturation_test): 필터 실행 모드에서 입력을 인가하고 내부 동작을 관측함.

6.2. 검증 시나리오 상세 분석

가. 계수 로딩(load_coefficients task)

- 목표
 - 필터 재구성(Reconfigurable) 기능의 핵심인 계수 쓰기 동작을 검증함.
- 동작
 - 테스트벤치 내부 **coefficients** 배열에 필터 계수를 미리 정의함.
 - **iCoeffUpdateFlag**를 1로 설정하여 FSM을 **p_Write** 상태로 진입시킴.
 - **for** 루프를 사용해 **iAddrRam**과 **iWrDtRam**을 제어하며 정의된 계수를 **SRAM**의 특정 뱅크·주소에 순차적으로 기록.
 - **\$display**를 활용하여 어떤 계수가 어느 주소에 기록되는지 출력함으로써 디버깅을 용이하게 함. (e.g. # [87500] Writing coeff[0]=500 to Addr=00)
 - 기록이 완료되면 **iCoeffUpdateFlag**를 0으로 내려 FSM을 **p_Idle** 상태로 되돌림.

나. 필터 동작 테스트(run_fir_test task)

- 목표
 - 로딩된 계수를 기반으로 **FIR** 필터링 연산이 올바르게 수행되는지 검증함.
- 입력 신호
 - 임펄스(Impulse) 입력을 사용함. 입력을 한 순간만 1(또는 -1)로 만들고 이후 0을 유지하는 방식이며, **FIR** 필터의 임펄스 응답은 계수(-1시에는 음양이 뒤바뀐)와 동일하므로 기능 검증에 가장 효과적임.
- 동작
 - 600kHz 샘플링 주기에 맞게 **iEnSample600k** 펄스를 주기적으로 생성함.
 - **iEnSample** 시점에 **iFirIn**에 1을 인가하고 복귀하여 임펄스 입력을 생성함.
 - **repeat** 루프를 통해 충분한 시간(필터 길이 이상의 클럭 주기) 동안 시뮬레이션을 진행하여 출력이 안정화되는 과정을 관측.
- 결과 확인
 - 이 테스트벤치는 시뮬레이션 종료 후 직접 파형을 보고 **oFirOut**이 임펄스 응답 형태로 계수와 동일하게 나타나는지를 값을 비교하여 분석함.

라. 필터 saturation 테스트(fir_saturation_test task)

- 목표
 - FIR 필터링 연산 과정에서 **saturation**이 올바르게 수행되는지 검증함.
- 입력 신호
 - 3배 Impulse 입력을 생성,
- 동작
 - 600kHz 샘플링 주기에 맞게 iEnSample600k 펄스를 주기적으로 생성함.
 - iEnSample 시점에 iFirIn에 3을 인가하고 복귀하여 3배 임펄스 입력을 생성함.
 - repeat 루프를 통해 충분한 시간(필터 길이 이상의 클럭 주기) 동안 시뮬레이션을 진행
 - 필터의 반주기 delay 후 다시 한 번 iFirIn에 3을 인가하고 복귀하여 3배 임펄스 입력을 생성함.
 - 16비트 양수 표현범위를 넘어서는 계산 결과 유발
- 결과 확인
 - 이 테스트벤치는 시뮬레이션 종료 후 직접 파형을 보고 oFirOut이 값을 보고 **saturation**이 제대로 동작하는지 검증함.

6.3. 기본 동작 결과

ReConf_FirFilter_tb를 사용하여 테스트 시나리오를 실행, waveform을 확인하고 작동을 검증

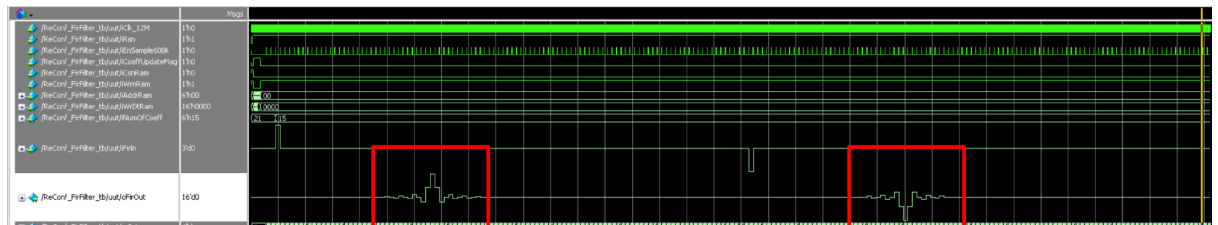
가. 출력값 분석을 통한 전체 필터 기본 작동 검증

- Impulse Response 관측



- iNumOfCoeff에 21 coefficient 입력(필터 길이 지정)
- Impulse response 가 coefficient 값과 같음을 확인 (FIR 필터 특성)
- 전체적인 흐름이 정상적으로 동작하여 저장, 모드 전환, 연산 제어 등이 이루어짐을 알 수 있음

- Impulse + negative impulse response 관측

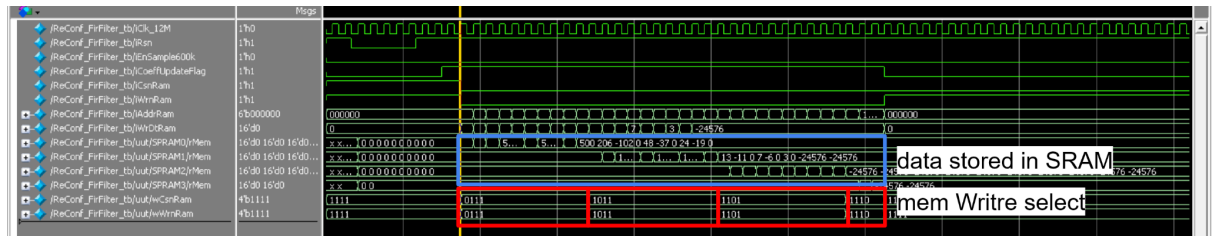


- iNumOfCoeff에 21 coefficient 입력(필터 길이 지정)
- negative Impulse response 가 negative coefficient 값과 같음을 확인
- signed input에서도 정상적으로 계산이 이루어짐을 알 수 있음
- 한 번의 입력이 아닌 추가적인 입력을 추가하여 연속적으로 이루어질 수 있음을 확인

6.4. 세부 waveform을 통한 검증

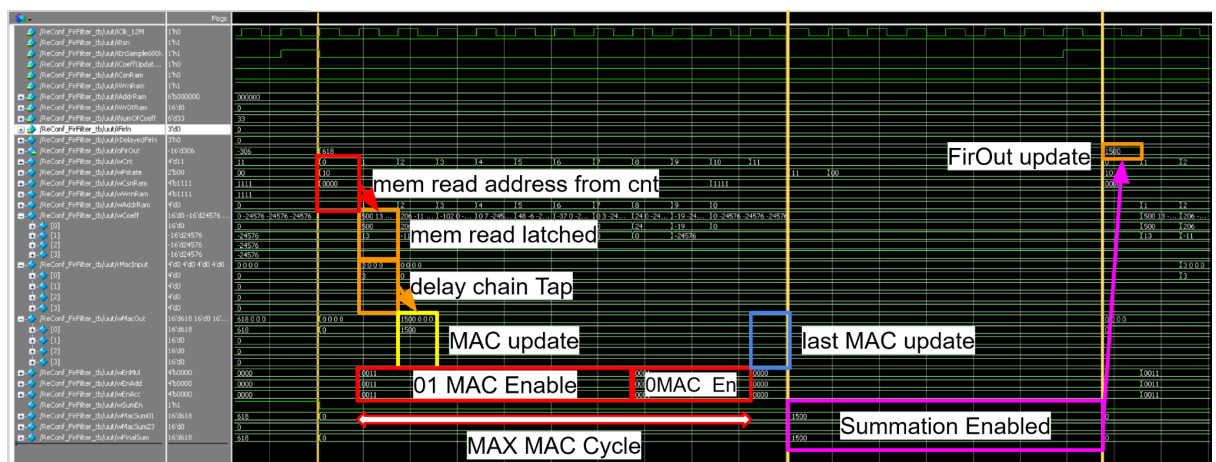
가. 세부 waveform 분석을 통한 내부 동작의 적절성 검증

● SRAM data 저장 과정 검증



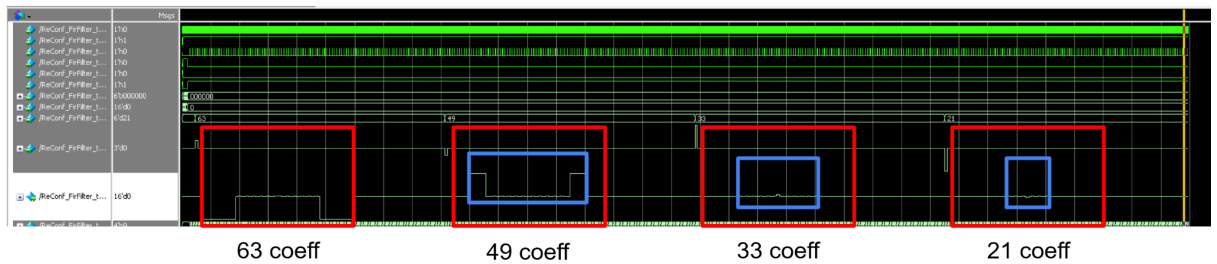
- write 상태에서 입력된 address에 맞게 상위 비트를 해석하여 적절한 SRAM에 Write 신호를 정상적으로 arbitration 함을 확인
- SRAM에 들어간 data가 정상적으로 SRAM 내부에 저장됨을 확인

● MAC 연산 과정 검증

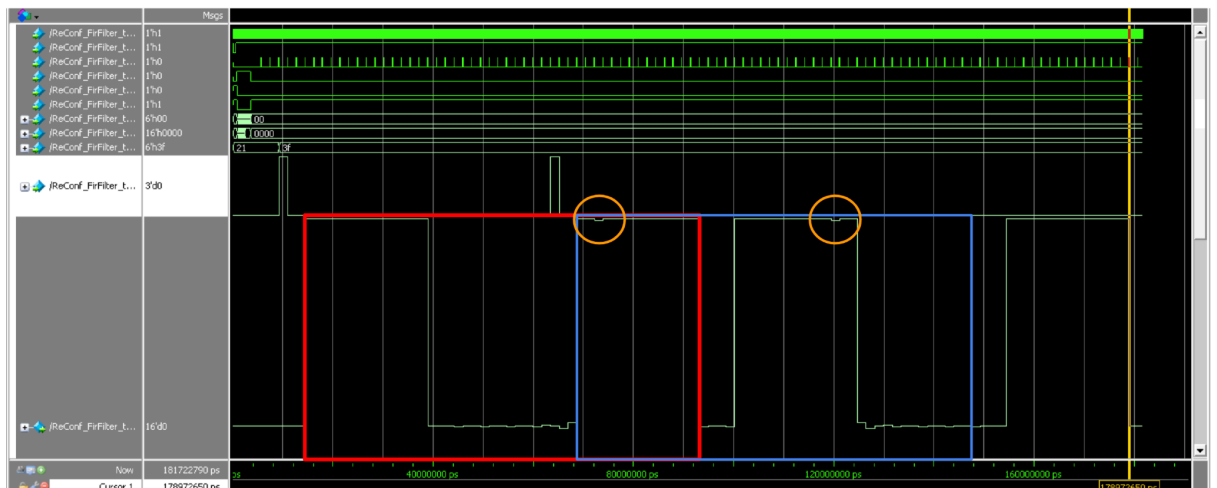


- counter를 address로 사용, SRAM에서 정상적으로 저장된 값이 읽어짐을 확인
- Delay chain값이 delay된 Coefficient와 짝이 맞게 불러짐을 확인
- ACC에 Multiply 결과가 정상적으로 accumulate됨을 확인
- MAC Enable 신호가 iNumOfCoeff와 맞는 정상적인 횟수로 동작함을 확인
($33 / 2 + 1 = 17, 10/7/0/0$)
- 또한 올바른 timing에 enable 되고 disable 됨을 확인
- 다음 sampling time에 값이 정상적으로 update됨을 확인

- 필터 길이가 가변 가능함을 검증



- dummy value(16'h7000)로 coefficient padding
- iNumOfCoeff를 바꾸며 각 필터 길이 입력에 맞게 연산됨을 관측
- 63 ~ 21개 길이로 모든 길이와 MAC에서 필터 연산이 동작함을 관측
- iNumOfCoeff를 사용하여 성공적으로 SRAM 속 쓰레기 값 차단이 가능함을 관측
- dummy value 입력으로 saturation이 정상적으로 동작함을 관측



- 계산 과정에서 **saturation**을 유발하여 확인
- Coeff SRAM에 **dummy value(16'h7000)** 값을 사용
- 3배 **impulse**를 발생시킴.
- 입력값인 3과 곱해진값은 16비트 양수 표현범위를 넘어서
- 출력값이 정상적인 양수 최대값이 나옴을 확인.
- (1) Mac 연산 내부에서의 **overflow**가 처리 됨을 확인.
- 두 번째 3배 **impulse**를 발생시켜 두 신호의 **response**가 겹치도록 한 후 겹친 부분의 계산값 관측.
- **impulse response**의 양수와 **MAX**와 더해진 구간에서 **MAX**가, 음수와 더해진 곳에서 **MAX + 음수** 연산이 정상적으로 수행됨을 관측
- (2) **summation logic**에서도 추가적으로 동작함을 확인

7. 구조 개선안

7.1. 현재 구조의 개선안

- **FSM 구조 개선**
 - FSM에 Main state 및 Counter가 복잡하게 얽혀있다. 이 때문에 state를 이해하고 state가 전환되는 부분의 code가 명확하게 설명되지 못하고 있어 개선할 수 있을 것이다.
 - 초기 구조에서 SUM과 FSM IDLE이 분리되어 있었으나, 실질적으로 IDLE에서도 Sum Enable 신호가 사용되면서 state reduction을 시도할 수 있을 것이다.
- **MAC controller 안전성 개선**
 - MAC controller가 iNumofCoeff를 wire로 받고 있는데, 이를 한 사이클 연산동안 값이 바뀌지 않도록 보장 할 수 있게 Flip Flop을 추가하여 한 사이클 연산 중 바뀌지 않도록 보장할 수 있을 것 같다.
- **MAC 및 SRAM 모듈 개수 축소**
 - 여기서 MAC이 현재 10클럭동안 10개의 coeff를 나눠서 계산하고, 12클럭에 연산이 완성된 후 20클럭 까지는 IDLE 상태이다. 이를 SRAM 주소 체계가 4비트 이고, MAC은 약 6~7 클럭 정도 연산의 여유가 있으므로 이를 활용하면 조금 더 큰 사이즈의 SRAM을 조합하면 많은 수의 트랜지스터를 아낄 수 있을것이다.
- **saturation logic 효율성 개선**
 - saturation을 처리하기 위해서 MAC내부에서 16비트보다 큰 bit 폭을 사용하고 saturation fixer를 통해서 핸들링 하고 있다. 이는 추가된 bit폭 만큼의 FF, 좀 더 큰 adder가 사용되면서 transistor가 낭비되고 있다고 볼 수 있다. 미리 MAC속 adder에서 saturation을 감지하고 handling 한다면 트랜지스터를 아낄 수 있을것이다.

8. 개인 project 소감

8.1. 김동준

소감)

앞선 교과목인 신호 및 시스템, 디지털 신호 처리, 디지털 공학 과목에서 배운 내용은 수식과 블록도같은 추상적인 개념을 다루어 막연했는데, 이번 프로젝트를 통해 블록도를 직접 HDL로 구현해보고, 구현된 FIR 필터에서 값이 나오는 모습을 보니 정말 신기했다.

익숙하지 않은 Verilog HDL을 다루는 과정에서 어려움이 많았다. 그동안 C 언어처럼 소프트웨어 관점에 익숙해져 있던 사고방식으로는 HDL의 방식을 받아들이는 과정이 처음엔 많이 낯설었다. 특히 sequential logic와 combinational logic의 타이밍 특성 때문에 timing이 엇나면서 예상치 못한 문제가 생겼을 때, code를 보는 것이 아닌 직접 파형을 하나하나 보면서 원인을 추적하는 과정이 힘들면서도 가장 많이 배운 부분이었다. 이 과정을 거치며 실제 회로의 동작을 이해하고 제대로 설계하는 것이 디지털 설계에서 얼마나 중요한지 몸으로 느낄 수 있었다.

8.2. 최영

소감)

FSM 설계 역할을 맡아 코드를 작성하는 과정에서 처음에는 혼자 해결하려다 보니 여러 어려움이 있었습니다. 이에 팀원들과 함께 제가 구상한 동작 방식과 다른 팀원들이 생각한 방식을 비교하며 논의를 진행했고, 그 과정에서 전체 시스템의 동작 원리를 명확하게 정리할 수 있었습니다. 의견을 조율하여 하나의 구조로 정리해냈을 때 상당한 성취감을 느꼈습니다.

물론 이를 실제 Verilog 코드로 구현하는 과정은 또 다른 도전이었습니다. 타이밍 제약과 입력 신호의 유효 시점을 고려해 FSM을 구성해야 했고, 시스템 전체가 자연스럽게 동작하도록 흐름을 설계해야 했습니다. 그러나 이러한 과정을 직접 수행하며 디지털 시스템 설계의 실질적인 난이도와 함께 그 속에서 느껴지는 보람을 크게 느낄 수 있었습니다.

8.3. 허준범

소감)

프로젝트에서 저는 메모리 컨트롤러와 연산 모듈 테스트를 맡아 **FIR** 필터가 안정적으로 동작하는지 검증했습니다. 여러 모듈이 서로 얽혀 있는 구조를 이해해야 해서 단순히 코드의 동작 여부만 보는 것이 아니라, 하드웨어 전체 데이터 흐름과 타이밍까지 함께 고려해야 했다는 점이 초반에는 특히 부담스럽고 어려웠습니다. 설계 문서와 파형을 번갈아 보며 구조를 머릿속에 그리는 과정이 오래 걸렸고, 이해가 조금만 어긋나도 바로 오동작으로 이어져 다시 처음부터 확인해야 하는 경우도 많았습니다.

또한 팀원들과 인터페이스 규격과 제어 신호 조건 등을 맞춰 가는 과정에서 설계와 코드를 여러 번 수정해야 했습니다. 그 과정은 힘들었지만, 문제를 구조적으로 쪼개서 원인을 좁혀 나가는 디버깅 방식에 익숙해질 수 있었고, 충분한 검증과 조율 끝에 **FIR** 필터가 의도대로 안정적으로 동작하는 모습을 확인했을 때 하드웨어 설계의 난이도와 그만큼의 보람을 동시에 느낄 수 있었습니다.

9. 별첨

9.1. inner wiring specification

MAC Unit	(wCnt)	address	wTap Index	Delay Chain Input A (Positive Offset)	Delay Chain Input B (Negative Offset)	Description	coeff_value
MAC 0	0	00_0000	wTap[0]	rDelayCain[31]	N/A	Center Tap (Middle)	500
MAC 0	1	00_0001	wTap[1]	rDelayCain[32]	rDelayCain[30]	Symmetric Pair 1	206
MAC 0	2	00_0010	wTap[2]	rDelayCain[33]	rDelayCain[29]	Symmetric Pair 2	-102
MAC 0	3	00_0011	wTap[3]	rDelayCain[34]	rDelayCain[28]	Symmetric Pair 3	0
MAC 0	4	00_0100	wTap[4]	rDelayCain[35]	rDelayCain[27]	Symmetric Pair 4	48
MAC 0	5	00_0101	wTap[5]	rDelayCain[36]	rDelayCain[26]	Symmetric Pair 5	-37
MAC 0	6	00_0110	wTap[6]	rDelayCain[37]	rDelayCain[25]	Symmetric Pair 6	0
MAC 0	7	00_0111	wTap[7]	rDelayCain[38]	rDelayCain[24]	Symmetric Pair 7	24
MAC 0	8	00_1000	wTap[8]	rDelayCain[39]	rDelayCain[23]	Symmetric Pair 8	-19
MAC 0	9	00_1001	wTap[9]	rDelayCain[40]	rDelayCain[22]	Symmetric Pair 9	0
MAC 1	0	01_0000	wTap[10]	rDelayCain[41]	rDelayCain[21]	Symmetric Pair 10	13

MAC 1	1	01_0001	wTap[11]	rDelayCain[42]	rDelayCain[20]	Symmetric Pair 11	
MAC 1	2	01_0010	wTap[12]	rDelayCain[43]	rDelayCain[19]	Symmetric Pair 12	
MAC 1	3	01_0011	wTap[13]	rDelayCain[44]	rDelayCain[18]	Symmetric Pair 13	
MAC 1	4	01_0100	wTap[14]	rDelayCain[45]	rDelayCain[17]	Symmetric Pair 14	
MAC 1	5	01_0101	wTap[15]	rDelayCain[46]	rDelayCain[16]	Symmetric Pair 15	
MAC 1	6	01_0110	wTap[16]	rDelayCain[47]	rDelayCain[15]	Symmetric Pair 16	
MAC 1	7	01_0111	wTap[17]	rDelayCain[48]	rDelayCain[14]	Symmetric Pair 17	
MAC 1	8	01_1000	wTap[18]	rDelayCain[49]	rDelayCain[13]	Symmetric Pair 18	
MAC 1	9	01_1001	wTap[19]	rDelayCain[50]	rDelayCain[12]	Symmetric Pair 19	
MAC 2	0	10_0000	wTap[20]	rDelayCain[51]	rDelayCain[11]	Symmetric Pair 20	
MAC 2	1	10_0001	wTap[21]	rDelayCain[52]	rDelayCain[10]	Symmetric Pair 21	
MAC 2	2	10_0010	wTap[22]	rDelayCain[53]	rDelayCain[9]	Symmetric Pair 22	
MAC 2	3	10_0011	wTap[23]	rDelayCain[54]	rDelayCain[8]	Symmetric Pair 23	
MAC 2	4	10_0100	wTap[24]	rDelayCain[55]	rDelayCain[7]	Symmetric Pair 24	
MAC 2	5	10_0101	wTap[25]	rDelayCain[56]	rDelayCain[6]	Symmetric Pair 25	
MAC 2	6	10_0110	wTap[26]	rDelayCain[57]	rDelayCain[5]	Symmetric Pair 26	

MAC 2	7	10_0111	wTap[27]	rDelayCain[58]	rDelayCain[4]	Symmetric Pair 27	
MAC 2	8	10_1000	wTap[28]	rDelayCain[59]	rDelayCain[3]	Symmetric Pair 28	
MAC 2	9	10_1001	wTap[29]	rDelayCain[60]	rDelayCain[2]	Symmetric Pair 29	
MAC 3	0	11_0000	wTap[30]	rDelayCain[61]	rDelayCain[1]	Symmetric Pair 30	
MAC 3	1	11_0001	wTap[31]	rDelayCain[62]	rDelayCain[0]	Edge Taps (First & Last)	